Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras

Due: November 6, 2026

Task

You are given a naive C implementation of single-head attention,

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V,$$

where Q, K, V and O are $N \times d$ row-major FP32 matrices and the softmax is applied to each row. Make it fast on your own machine, step by step, and explain every result with measurements.

Starter code

Create your own repository from the starter code:

`https://github.com/NMJ0/Assignment-5-Systems-for-Deep-learning`

Questions

Q1. Your machine (20 marks)

Fill in `machine_spec.txt`: CPU model, cores (physical, logical, performance and efficiency), clock frequencies, SIMD width and FMA units, cache sizes, memory type and size. From these, compute the peak FP32 GFLOP/s (one core and all cores), the memory bandwidth and the ridge point. Show your calculations in the report.

Q2. Naive baseline (20 marks)

Run `make naive`. For each input, report the time, GFLOP/s and percentage of the peak. How does the time change with N and with d? Explain where the time goes.

Q3. Optimisations (20 marks)

Write each variant, run it, and report the same measurements as Q2 plus the speedup over the naive variant. `make check` must pass. Explain what each change does and what limits performance afterwards. Do not use BLAS or other math libraries.

(a) `attn_loops.c`: reorder the loops so the compiler can vectorise them.

(b) `attn_tiled.c`: cache and register tiling.

(c) `attn_fused.c`: fuse the three steps row by row so the $N \times N$ matrix is never stored.

(d) `attn_best.c`: anything you want. Get the maximum single-threaded speedup you can, and explain what you did and why it works.

Q4. OpenMP (20 marks)

Speed up your best variant with OpenMP in `attn_omp.c`. Run it with different thread counts and report the time, speedup and parallel efficiency. Explain the scaling you see.

Q5. Roofline (20 marks)

Plot the roofline of your machine using your Q1 numbers, and place every variant on it at $N = 4096$, $d = 64$. Explain how you computed each variant's arithmetic intensity and what limits each variant.

Submission

Submit a link to a **public GitHub repository** with exactly this structure:

Path	Contents
`src/`	`attn_naive.c` and all your variants
`input/`	The given inputs
`machine_spec.txt`	Fill in the empty fields
`results.csv`	Fill in the empty fields (add rows for the thread counts you tried)
`plots/roofline.png`	The roofline with all variants
`plots/time_N<N>_d<d>.png`	One plot per input: time of every variant
`Makefile`	Makefile
`assignment_handout.pdf`	This handout
`report.pdf`	Your answers to Q1–Q5